STEP 1 — Install Required Libraries

```
!pip install torch torchvision scikit-learn pandas numpy matplotlib
```

```
Requirement already satisfied: torch in /usr/local/lib/python3.12/dist-packages (2.10.0+cpu)
Requirement already satisfied: torchvision in /usr/local/lib/python3.12/dist-packages (0.25.0+cpu)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.6.1)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (2.0.2)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch) (3.25.2)
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from torch) (4.15.0)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch) (3.6.1)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from torch) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-packages (from torch) (2025.3.0)
Requirement already satisfied: pillow!=8.3.*,>=5.3.0 in /usr/local/lib/python3.12/dist-packages (from torchvision) (11.3.0)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.16.3)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (3.6.0)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.3)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.62.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.5.0)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (26.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.2)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas) (1.17.0
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->torch) (3.0.3)
```

STEP 2 — Import Libraries

```
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import torch.optim as optim

from sklearn.datasets import fetch_20newsgroups
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

import re
from collections import Counter
import matplotlib.pyplot as plt
```

STEP 3 — Load Dataset

```
categories = ['rec.sport.hockey', 'sci.space']

data = fetch_20newsgroups(subset='all', categories=categories)

texts = data.data
labels = data.target

print("Total samples:", len(texts))
```

```
Total samples: 1986
```

STEP 4 — Text Preprocessing

```
def preprocess(text):
    text = text.lower()
    text = re.sub(r'[^a-z\s]', '', text)
    return text

texts = [preprocess(t) for t in texts]
```

STEP 5 — Tokenization + Vocabulary

```
tokenized_texts = [t.split() for t in texts]

# Build vocabulary
counter = Counter()
for tokens in tokenized_texts:
    counter.update(tokens)

vocab = {word: i+1 for i, (word, _) in enumerate(counter.most_common(10000))}
vocab_size = len(vocab) + 1

print("Vocab size:", vocab_size)
```

```
Vocab size: 10001
```

STEP 6 — Encoding & Padding

```
max_len = 100

def encode(tokens):
    return [vocab.get(word, 0) for word in tokens][:max_len]

encoded_texts = [encode(t) for t in tokenized_texts]

# Padding
def pad(seq):
    return seq + [0]*(max_len - len(seq))

encoded_texts = np.array([pad(seq) for seq in encoded_texts])
labels = np.array(labels)
```

STEP 7 — Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    encoded_texts, labels, test_size=0.2, random_state=42
)
```

STEP 8 — PyTorch Dataset Class

```
from torch.utils.data import Dataset, DataLoader

class TextDataset(Dataset):
    def __init__(self, X, y):
        self.X = torch.LongTensor(X)
        self.y = torch.LongTensor(y)

    def __len__(self):
        return len(self.X)

    def __getitem__(self, idx):
        return self.X[idx], self.y[idx]

train_dataset = TextDataset(X_train, y_train)
test_dataset = TextDataset(X_test, y_test)

train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=32)
```

STEP 9 — Build 1D CNN Model

```
class CNN_Text(nn.Module):
    def __init__(self, vocab_size, embed_dim=100):
        super(CNN_Text, self).__init__()

        self.embedding = nn.Embedding(vocab_size, embed_dim)

        self.conv = nn.Conv1d(
            in_channels=embed_dim,
            out_channels=128,
```

```
            kernel_size=5
        )

        self.pool = nn.MaxPool1d(2)

        self.fc = nn.Linear(128*48, 2)  # adjust size if needed

        self.relu = nn.ReLU()

    def forward(self, x):
        x = self.embedding(x)            # (batch, seq, embed)
        x = x.permute(0, 2, 1)           # (batch, embed, seq)
        x = self.conv(x)
        x = self.relu(x)
        x = self.pool(x)
        x = x.view(x.size(0), -1)
        x = self.fc(x)
        return x

model = CNN_Text(vocab_size)
```

STEP 10 — Training Setup

```
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)
```

STEP 11 — Train Model

```
epochs = 5

for epoch in range(epochs):
    model.train()
    total_loss = 0

    for X_batch, y_batch in train_loader:
        optimizer.zero_grad()

        outputs = model(X_batch)
        loss = criterion(outputs, y_batch)

        loss.backward()
        optimizer.step()

        total_loss += loss.item()

    print(f"Epoch {epoch+1}, Loss: {total_loss:.4f}")
```

```
Epoch 1, Loss: 28.2361
Epoch 2, Loss: 5.4529
Epoch 3, Loss: 1.2616
Epoch 4, Loss: 0.5399
Epoch 5, Loss: 0.3047
```

STEP 12 — Evaluation

```
model.eval()

y_pred = []
y_true = []

with torch.no_grad():
    for X_batch, y_batch in test_loader:
        outputs = model(X_batch)
        _, predicted = torch.max(outputs, 1)

        y_pred.extend(predicted.numpy())
        y_true.extend(y_batch.numpy())

print("Accuracy:", accuracy_score(y_true, y_pred))
print("\nClassification Report:\n", classification_report(y_true, y_pred))
```

```
Accuracy: 0.9623115577889447
```

```
Classification Report:
              precision    recall  f1-score   support

           0       0.96      0.97      0.96       202
           1       0.97      0.95      0.96       196

    accuracy                           0.96       398
   macro avg       0.96      0.96      0.96       398
weighted avg       0.96      0.96      0.96       398
```

STEP 13 — Confusion Matrix

```
import seaborn as sns

cm = confusion_matrix(y_true, y_pred)

sns.heatmap(cm, annot=True, fmt='d')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```